

OPTIMIZING ALGORITHMS FOR QUERIES

SQLDeveloper and APEX have a tool for analyzing the time and the execution algorithms for queries – *Explain Plan*. When a query is executed, depending on several facts, Oracle uses a certain algorithm, usually an optimized one. The working principle is based on the cost of executing the query by applying all the various optimization algorithms based on statistics obtained from the metadata dictionary, that are related to: relational operators, number of records in each table involved, indexes, clusters and partitions available for those tables involved in the query. There are taken into account other factors such as data access paths, joins order, available physical resources (CPU, memory, I / O operations). It is chosen the algorithm that minimizes the cost of implementation and involves a minimum use of resources. These algorithms are selected by default by Oracle, but can be chosen by the programmer using directives inserted into SQL, in order to set the OPTIMIZER_MODE parameter. A query can be executed in many ways, applying various optimization algorithms, such as: full table scans, index scans, nested loops, hash joins, sort merge joins. The algorithms can be applied to different sets of records, by specifying in the directive that it will be executed for the first n records (FIRST_ROWS (n)) or for all entries (ALL_ROWS).

Without using the optimization algorithms, the execution cost is:

```
select
d.department_name
, e.first_name
, e.last_name
, e.salary
from employees e, departments d
where d.department_id=e.department_id
```

SQLDeveloper

Explain Plan x

SQL | 2.289 seconds

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			106	6
MERGE JOIN			106	6
TABLE ACCESS	DEPARTMENTS	BY INDEX ROWID	27	2
INDEX	DEPT_ID_PK	FULL SCAN	27	1
SORT		JOIN	107	4
Access Predicates	D.DEPARTMENT_ID=E.DEPARTMENT_ID			
Filter Predicates	D.DEPARTMENT_ID=E.DEPARTMENT_ID			
TABLE ACCESS	EMPLOYEES	FULL	107	3

APEX

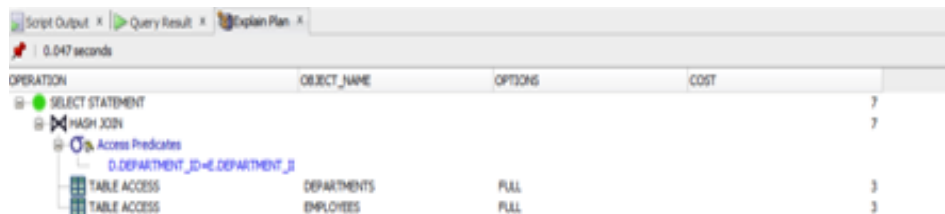
Results Explain Describe Saved SQL History

Operation	Options	Object	Rows	Time	Cost	Bytes	Filter Predicates *
SELECT STATEMENT			106	1	6	8,692	
MERGE JOIN			106	1	6	8,692	
TABLE ACCESS	BY INDEX ROWID	DEPARTMENTS	27	1	2	810	
INDEX	FULL SCAN	DEPT_ID_PK	27	1	1		
SORT	JOIN		107	1	4	5,564	"D"."DEPARTMENT_ID" = "E"."DEPARTMENT_ID"
TABLE ACCESS	STORAGE FULL	EMPLOYEES	107	1	3	5,564	"D"."DEPARTMENT_ID"

1. Hash joins algorithm - is used for the queries that involve *tables with many records and an equijoin*. The algorithm consists in choosing the *smallest table* and building a *hash memory table* based on the join condition. Then, the other table is scanned for finding the corresponding records. This algorithm is used when the smallest table fits the internal memory and the disk access operations are minimized. The execution cost is limited to the time spent with accessing the larger table for the corresponding records.

In the following example, Departments table has 27 records and will be used as hash memory table, while Employees table will be scanned for its 107 records.

```
select /*+ USE_HASH(d e)*/
d.department_name
,e.first_name
,e.last_name
,e.salary
from employees e, departments d
where d.department_id=e.department_id
```



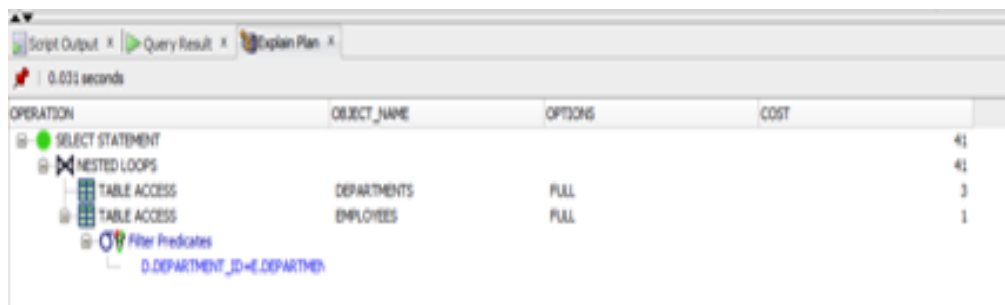
Script Output | Query Result | Explain Plan

0.047 seconds

OPERATION	OBJECT_NAME	OPTIONS	COST
SELECT STATEMENT			7
HASH JOIN			7
Access Predicates			
D.DEPARTMENT_ID=E.DEPARTMENT_ID			
TABLE ACCESS	DEPARTMENTS	FULL	3
TABLE ACCESS	EMPLOYEES	FULL	3

2. Nested loop joins algorithm – is recommended for joins between *relatively small data sets*. USE_NL has to be specified.

```
select /*+ USE_NL (d e)*/
d.department_name
,e.first_name
,e.last_name
,e.salary
from employees e, departments d
where d.department_id=e.department_id
```



Script Output | Query Result | Explain Plan

0.031 seconds

OPERATION	OBJECT_NAME	OPTIONS	COST
SELECT STATEMENT			41
NESTED LOOPS			41
TABLE ACCESS	DEPARTMENTS	FULL	3
TABLE ACCESS	EMPLOYEES	FULL	1
Filter Predicates			
D.DEPARTMENT_ID=E.DEPARTMENT_ID			

By comparing the two methods, the execution cost for Hash Joins is 7, while for Nested Lops is 41. In this case, the recommended method is Hash Joins.

3. Sort merge joins algorithm – is suitable for joins if *one of the tables has the records ordered*. In general, if a prior data sorting is done, this algorithm leads to lower execution costs compared to similar results obtained by applying the hash joins algorithm. Sort merge joins is *also recommended for non-equi joins or for very large data sets*. The execution principle is the following:

- Order data from both tables using the searching key (condition).
- Join the two ordered tables.

The choice of this algorithm is recommended for large data sets and conditions of inequality between tables because this type of join requires ordering data.

```
create index idx_emp_department_id on employees(department_id);
--create index idx_dep_department_id on departments(department_id);
```

```
select /*+ USE_MERGE (d e)*/
d.department_name
,e.first_name
,e.last_name
,e.salary
from employees e, departments d
where d.department_id=e.department_id
```

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			106	6
MERGE JOIN			106	6
TABLE ACCESS	DEPARTMENTS	BY INDEX ROWID	27	2
INDEX	DEPT_ID_PK	FULL SCAN	27	1
SORT		JOIN	107	4
Access Predicates				
D.DEPARTMENT_ID=E.DEPARTMENT_ID				
Filter Predicates				
D.DEPARTMENT_ID=E.DEPARTMENT_ID				
TABLE ACCESS	EMPLOYEES	FULL	107	3

In this case, the execution cost is 6, the minimum cost obtained by using the above methods.

4. Outer Joins algorithms - are used for outer joins: *Nested Loop Outer Joins, Hash Join Outer Joins, Sort Merge Outer Joins, Full Outer Joins*.

- *Nested Loop Outer Joins* – is used for outer joins and it works like this: one of the tables is chosen as pivot table, and the records from the other table are accessed in a repetitive cycle correlated with the join condition.

```
select /*+ USE_NL(d e)*/
d.department_id
,d.department_name
,e.first_name
```

```
,e.last_name
, e.salary
from employees e, departments d
where d.department_id=e.department_id(+)
```

If we put an additional condition for limiting the values for the department_id between 150 and 200, improved performances will be obtained:

```
select /*+ USE_NL(d e)*/
d.department_id
,d.department_name
, e.first_name
,e.last_name
, e.salary
from employees e, departments d
where d.department_id=e.department_id(+)
AND d.department_id between 150 and 200;
```

- *Hash Join Outer Joins* – is used for large volumes of data, so the Hash method becomes efficient and there is no need for a pivot table.

```
select /*+ USE_HASH (d e)*/
d.department_id
,d.department_name
, e.first_name
,e.last_name
, e.salary
from employees e, departments d
where d.department_id=e.department_id(+)
AND d.department_id between 150 and 200
```

- *Sort Merge Outer Joins* – is applied when no table can be chosen as a pivot table or when the existing conditions lead to cost growth when using the Hash algorithm or when the records are ordered.

```
SELECT /*+ USE_MERGE(d e) */
d.department_id
,d.department_name
, e.first_name
,e.last_name
, e.salary
from employees e, departments d
where d.department_id=e.department_id(+)
AND d.department_id between 150 and 200;
```

- *Full Outer Joins* – is used as an extension of left and right outer joins. It is applied after an inner join has been executed and adds all the records that were not selected from both tables including the null values.